EduFusion AI: An Intelligent Microservices and Agentic AI Framework for Smart Campus Management

Ch. Bhargavi Devi, S. Anjana, B. V. S. Jayadeep

Department of Computer Science and Engineering, KLH University, Hyderabad, India
2420030588@klh.edu.in; 2420030499@klh.edu.in; 2420030381@klh.edu.in

Abstract—Universities operate multiple digital functions for students, faculty, academic records, attendance, resources, events, transport, infrastructure, notifications, and administration. When these functions are implemented as isolated applications, users must manually combine information before making decisions. EduFusion AI addresses this problem through a role-aware React portal, Spring Boot microservices, PostgreSQL persistence, Apache Kafka event communication, and Python FastAPI intelligence services. The implementation contains twelve domain microservices in addition to an API Gateway and dashboard service, while the frontend provides separate experiences for Student, Faculty, HOD, Admin, and Parent roles. The current AI prototype contains a LangGraph intent-routing workflow and a student-risk prediction service. Its offline training script generates 1,000 synthetic student records and trains a 100-tree Random Forest classifier, obtaining 88.5% test accuracy on an 80:20 split. The deployed prediction endpoint currently uses a deterministic weighted risk function because the runtime implementation does not load the trained Random Forest model; this distinction is explicitly reported to avoid overstating experimental results. Prototype UI validation demonstrates integrated student management, attendance, academics, resources, events, notifications, and AI-insight views. The paper presents the architecture, dataset, implementation, measured prototype results, and a comparison with the 2025 conference paper CAMPUS-ONE.

Index Terms—smart campus, microservices, Apache Kafka, LangGraph, machine learning, student-risk prediction, role-based access control, AI assistant, campus management.

I. INTRODUCTION

Higher-education institutions continuously generate operational data from academic activities, attendance, student records, faculty work, classrooms, resources, events, transport, maintenance, and notifications. A portal that merely displays these records is useful for information access, but it does not automatically connect an event in one domain to a decision in another. For example, an attendance decline may require academic context, a faculty intervention, a notification, and later outcome monitoring. Similarly, an event can affect classroom allocation, resource demand, transport capacity, and campus communication.

EduFusion AI is implemented as a smart-campus software platform intended to reduce this fragmentation. Its frontend is a React/Vite single-page application with role-specific routing. The backend is decomposed into independently deployable Spring Boot services, with PostgreSQL used for persistence and Apache Kafka used for asynchronous event communication. Python FastAPI services provide prediction and agent capabilities. This separation is consistent with the objective of making individual campus domains independently testable and scalable.

The contribution of this work is therefore not only a dashboard. It is the integration of a campus portal, domain microservices, event-driven communication, predictive analytics, and an agentic interface. Importantly, the present prototype is distinguished from a fully autonomous multi-agent production system: the implemented LangGraph workflow currently performs intent parsing, routing, student/attendance actions, prediction calls, and response generation. The paper reports this implementation boundary explicitly.

II. LITERATURE SURVEY

Recent conference research shows a rapid movement toward AI-supported educational platforms. CAMPUS-ONE, published in the 2025 6th International Conference on IoT Based Control Networks and Intelligent Systems, presents an AI agent for institutional support [1]. Its focus is closely related to EduFusion AI because both position an AI agent as an institutional support mechanism. EduFusion AI extends the implementation focus toward a complete role-based campus portal with multiple backend services and event-driven infrastructure.

SAPP, presented at the 2025 IEEE Global Engineering Education Conference, targets student academic-risk prediction across diverse modules. Its motivation is particularly relevant to EduFusion AI's student-risk component because it connects prediction with early intervention [2]. Other 2025 IEEE conference work has explored AI-IoT-enabled smart-campus environments, emphasizing interoperability, modularity, resource usage, and student support [3]. These studies indicate that prediction and intelligent campus services are increasingly being evaluated as integrated systems rather than isolated algorithms.

Conference work on academic assistants and student-performance prediction further demonstrates the value of conversational support and machine learning for education [4]–[6]. EduFusion AI combines these directions with microservice decomposition and event communication. Apache Kafka is used in the implementation as the event backbone, while the AI service is kept outside the Java service boundary so that model and orchestration components can evolve independently.

III. RESEARCH GAP AND CONTRIBUTIONS

TABLE I. RESEARCH GAP AND IMPLEMENTATION-LEVEL POSITIONING

Dimension	Existing direction	EduFusion AI implementation
Institutional support	AI assistant / institutional support	Role-based campus portal plus AI assistant
Backend organization	Application-level integration	12 domain microservices + API Gateway
Event communication	Not central to every workflow	Kafka producer/consumer pattern
Prediction	Academic prediction or analytics	Student risk prediction service + dashboard
User roles	Often single interaction model	Student, Faculty, HOD, Admin, Parent
Operational modules	Focused institutional tasks	Academics, attendance, resources, events, transport
AI orchestration	Agent-centric interaction	LangGraph intent routing connected to prediction/service actions

The principal research gap addressed by EduFusion AI is the lack of a unified implementation in which heterogeneous campus services are exposed through a common role-aware interface and can exchange events with AI-enabled services. The system contributes: (1) domain microservice decomposition; (2) role-based portal routing; (3) Kafka-based event propagation; (4) a measurable student-risk prediction pipeline; and (5) an agentic FastAPI interface that can route natural-language requests toward campus functions.

Compared with CAMPUS-ONE, EduFusion AI is stronger in breadth of implemented campus operations and in explicit separation between frontend roles, domain services, event infrastructure, and AI services. However, CAMPUS-ONE should not be judged as weaker on AI quality without its full implementation being available; the comparison in this paper is architectural and implementation-oriented rather than a claim that one system has universally superior intelligence.

IV. PROPOSED EDUFUSION-AI ARCHITECTURE

The architecture follows four logical levels: presentation, domain microservices, event coordination, and AI services. PostgreSQL provides persistent campus state. Spring Boot services expose REST endpoints. Apache Kafka carries asynchronous events. The Python services provide prediction and agent functionality. The React portal consumes authorized backend APIs and presents different capabilities according to the authenticated role.

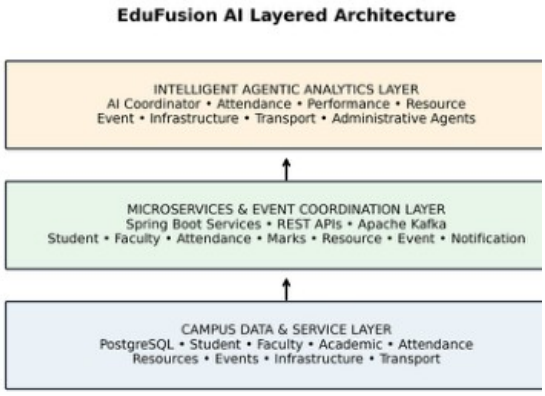


Fig. 1. Layered architecture of EduFusion AI.

A. Role-Based Presentation Layer

The frontend implements protected routes for five roles: STUDENT, FACULTY, HOD, ADMIN, and PARENT. The routing layer redirects users to the dashboard associated with their role and rejects unauthorized role access. Student pages include academics, attendance, marks, events, resources, notifications, AI insights, and profile. Faculty pages add student management, teaching, performance, attendance, leave, and resource/event operations. HOD and Admin receive broader analytical and operational views, while Parent receives child and progress information.

B. Domain Microservices

TABLE II. IMPLEMENTED SPRING BOOT SERVICE LANDSCAPE

Service	Primary responsibility
API Gateway	Central routing and entry point
Auth	Authentication and user management
Student	Student records and profiles
Faculty	Faculty operations
Academic	Courses and curriculum
Classroom	Classroom/room management
Attendance	Attendance tracking
Dashboard	Aggregated dashboard data
Resource	Campus resources
Transport	Campus transport information
Event	Campus event management
Infrastructure	Maintenance/infrastructure requests
Notification	Notification consumption and dispatch

C. Event Coordination Layer

The event-service contains a Kafka publisher and the notification-service contains a Kafka listener. The current implementation therefore demonstrates the basic producer-consumer path: an event can be published to the notification topic and consumed asynchronously by the notification service. This provides a concrete foundation for extending attendance, event, resource, and infrastructure workflows.

D. AI Services

Two Python FastAPI services are present. The prediction service exposes /predict/student-performance. The request contains student_id, attendance_rate, assignment_score, exam_score, and engagement_level. The response returns risk_level, predicted_final_score, and confidence. The agent service exposes /agent/chat and implements a LangGraph StateGraph with intent parsing, conditional routing, action nodes, and response generation. Prediction queries can call the live prediction endpoint.

lministrative Agent. The Coordinator determines which ag
ɔuld participate in a cross-service task.

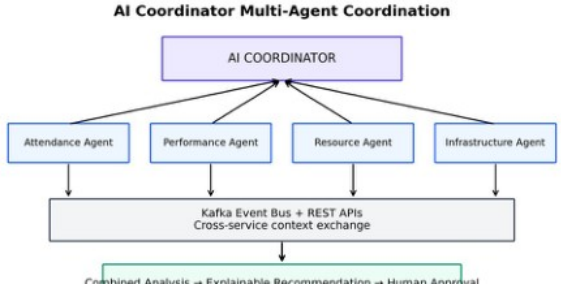


Fig. 2. AI coordinator and agent-routing concept used to organize cross-service intelligence.

V. METHODOLOGY

A. Campus Data Flow

A typical request follows the sequence: authenticated user → React role portal → API Gateway/service endpoint → domain microservice → PostgreSQL. When asynchronous coordination is required, the producing service emits an event to Kafka and the relevant consumer processes the message. AI requests follow a separate path to the FastAPI agent or prediction service. This separation prevents the frontend from depending directly on internal database structures.

B. Student-Risk Prediction

The offline training script generates four input features: attendance rate, assignment score, exam score, and engagement level. The synthetic target is generated from a weighted risk score: 0.3 attendance + 0.2 assignment + 0.4 examination + 0.1 engagement, after normalization. Risk is HIGH below 0.65, MEDIUM from 0.65 to below 0.80, and LOW at 0.80 or above. A Random Forest classifier with 100 estimators is trained using an 80:20 train-test split with random_state=42.

C. Deployed Inference Logic

The deployed prediction endpoint currently reproduces the same weighted decision logic directly in Python rather than loading the saved Random Forest model. This is an implementation constraint and is explicitly separated from the offline model result. The final-score field is calculated as 0.4 × assignment score + 0.6 × examination score. Confidence is currently a rule-based response field (0.89 for HIGH, 0.75 for MEDIUM, and 0.92 for LOW), so it must not be interpreted as calibrated probability.

D. Agentic Workflow

The LangGraph state contains prompt, intent, action_result, and response. Intent parsing recognizes login/help, student queries, prediction queries, attendance queries, and greetings. Student and attendance intents route to a service-action node, while prediction intents route to the prediction service. The final node converts the action result into a role-oriented natural-language response. This is a practical agentic workflow, but it is not yet a fully autonomous multi-agent architecture; additional specialized agents can be added without changing the service boundaries.

E. Human Control and Security

Role-based routing is enforced in the React application, while production-grade authorization must remain enforced by backend services. JWT authentication, service-level authorization, password hashing, audit logging, and restricted administrative operations are appropriate controls for deployment. AI tools should access narrowly scoped APIs rather than unrestricted database connections.

VI. DATASET

The current predictive experiment uses a synthetic dataset generated by the project training script rather than a real institutional dataset. It contains 1,000 records generated with NumPy seed 42. Attendance is sampled from 0.50–1.00, assignment score from 40–100, exam score from 40–100, and engagement from 0–10. The target is derived from the weighted rule described in Section V. The resulting class distribution is 341 HIGH, 515 MEDIUM, and 144 LOW records.

The frontend additionally contains demonstration values for dashboards. These values are useful for functional validation but are not treated as institutional measurements. For example, the faculty performance view displays 128 students taught, three subjects, 85% average attendance, and 82% average marks. The student-management view shows sample attendance values of 92%, 71%, 88%, and 78% for four visible records. These are reported as UI demonstration outputs only.

TABLE III. SYNTHETIC PREDICTION DATASET

Feature	Generation / range	Role
attendance_rate	Uniform 0.50–1.00	Attendance signal
assignment_score	Uniform 40–100	Continuous assessment
exam_score	Uniform 40–100	Examination performance
engagement_level	Uniform 0–10	Engagement proxy
risk_level	Rule-derived HIGH/MEDIUM/LOW	Prediction target

VII. IMPLEMENTATION

A. Technology Stack

TABLE IV. IMPLEMENTED TECHNOLOGY STACK

Layer	Implemented technology
Frontend	React 19, Vite, React Router, Recharts, Framer Motion, Lucide
Backend	Java 21, Spring Boot, Maven
API Gateway	Spring Cloud Gateway
Database	PostgreSQL 15
Messaging	Apache Kafka
AI API	Python, FastAPI, Uvicorn
Agent orchestration	LangGraph
ML	scikit-learn, Random Forest training script
Deployment	Docker Compose; Kubernetes manifests included

B. Functional Portal Coverage

The implementation demonstrates a substantial set of campus-management screens. Faculty-facing views include student management, teaching and performance analytics, attendance logs, leave management, academics, events, resources, notifications, AI insights, and profile. Student-facing views include dashboard, academics, attendance, marks, events, resources, notifications, AI assistant, and profile. The same shared components are reused in HOD and Admin portals with different route permissions.

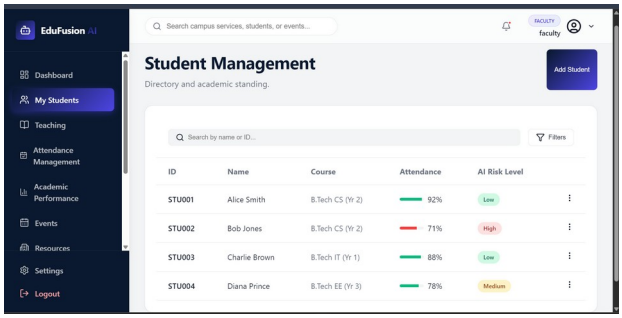


Fig. 4. Faculty student-management interface showing attendance and AI risk levels.

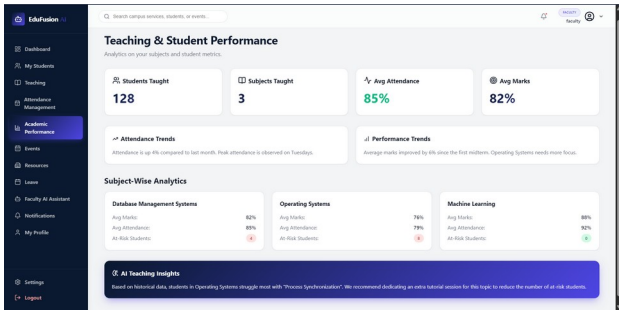


Fig. 5. Faculty teaching and student-performance analytics interface.

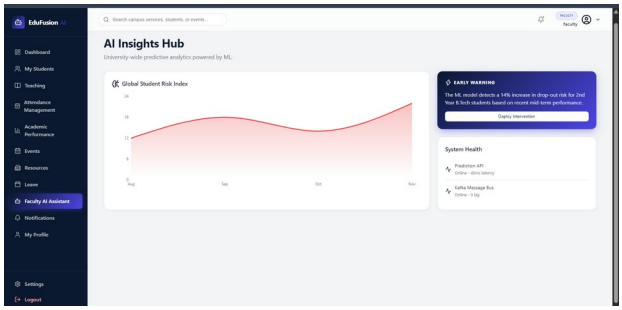


Fig. 6. AI Insights Hub with risk trend and early-warning visualization.

VIII. RESULTS AND EVALUATION

The results are divided into model-level measurements and prototype-level functional validation. This distinction is important because some dashboard numbers are intentionally illustrative, whereas the Random Forest result is computed from the reproducible training script. No dashboard value is presented as a real university measurement.

TABLE V. OFFLINE RANDOM FOREST RESULTS

Metric	Result	Interpretation
Training samples	1,000	Synthetic records
Train/test split	80/20	800 training, 200 test
Random Forest	100 trees	random_state=42
Test accuracy	88.5%	Offline classifier
Weighted precision	88.56%	Test-set weighted
Weighted recall	88.50%	Test-set weighted
Weighted F1	88.49%	Test-set weighted

TABLE VI. RANDOM FOREST CONFUSION MATRIX (200-RECORD TEST SET)

Actual \ Pred.	HIGH	MEDIUM	LOW
HIGH	68	7	0
MEDIUM	9	85	2
LOW	0	5	24

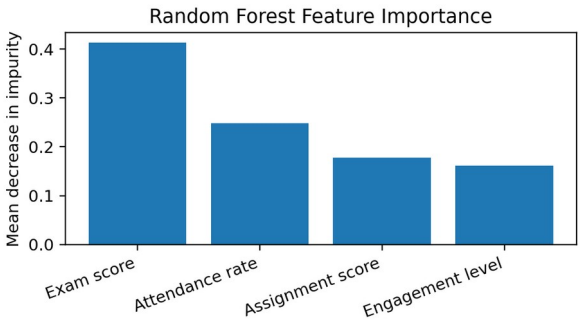


Fig. 7. Feature importance from the offline Random Forest training experiment.

The model obtained 88.5% accuracy on the fixed 200-record test set. Exam score contributed the largest feature importance (approximately 0.413), followed by attendance rate (0.248), assignment score (0.178), and engagement level (0.162). These values should be interpreted cautiously because the target labels themselves were generated using a weighted rule that gives examination score the largest coefficient.

A. Deployed Prediction Example

TABLE VII. EXAMPLE OUTPUT FROM THE DEPLOYED PREDICTION LOGIC

Input / output field	Value
student_id	1
attendance_rate	0.60
assignment_score	55
exam_score	60
engagement_level	4.0
weighted risk score	0.57
risk_level	HIGH
predicted_final_score	58.0
reported confidence	0.89

For the example request used by the agent service, the weighted score is 0.57, which maps to HIGH risk. The final-score calculation returns 58.0. The confidence value of 0.89 is a fixed rule-based field rather than a calibrated probability. Therefore, the offline 88.5% Random Forest result and the deployed endpoint output must be reported as two different evaluation artifacts.

B. Functional Dashboard Validation

TABLE VIII. UI VALIDATION VALUES OBSERVED IN THE IMPLEMENTED PROTOTYPE

Observed prototype output	Value
Faculty students taught	128
Subjects taught	3
Average attendance	85%
Average marks	82%
Visible student attendance range	71–92%
Visible attendance log records	4
Visible attendance status	3 Present, 1 Absent

The screenshots also demonstrate that the frontend renders role-specific information rather than a single generic dashboard. The Student Management screen exposes student identity, course, attendance, and AI risk level; the Teaching screen aggregates students and subjects; the Attendance screen provides event-level records; and the AI Insights Hub visualizes risk trends and an early-warning card. These results validate the presentation and integration layer, but they are not a substitute for production performance testing.

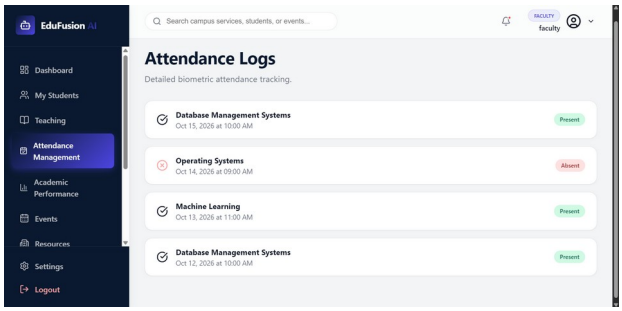


Fig. 8. Attendance log interface used for functional validation.

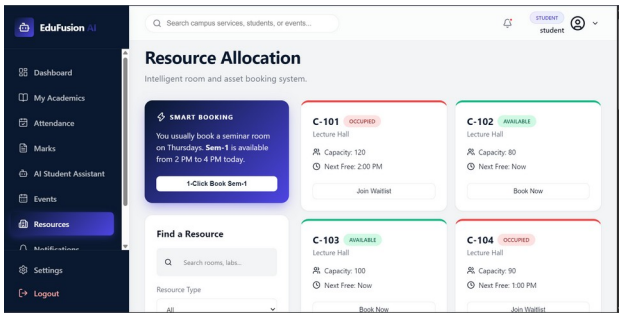


Fig. 9. Resource-allocation interface demonstrating campus resource management.

C. Event-Driven Validation

The event-service publishes messages through Kafka and the notification-service consumes the notification topic. This confirms the presence of an asynchronous service boundary in the implementation. A production evaluation should additionally measure event latency, consumer lag, duplicate handling, failed-message recovery, and end-to-end notification delivery. The present project does not yet provide sufficient repeated benchmark runs to report those values honestly.

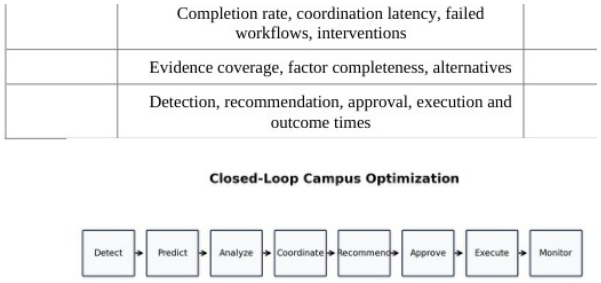


Fig. 10. Closed-loop decision-support concept used as the target workflow for future end-to-end evaluation.

IX. COMPARISON WITH CAMPUS-ONE

TABLE IX. IMPLEMENTATION-LEVEL COMPARISON WITH CAMPUS-ONE

Dimension	CAMPUS-ONE [1]	EduFusion AI
Publication	2025 ICICNIS conference paper	Current prototype / implementation
Core idea	AI agent for institutional support	AI-enabled smart-campus platform
User experience	Institutional AI-agent orientation	Separate Student, Faculty, HOD, Admin, Parent
Backend	Agent-centered paper implementation	12 Spring Boot domain services + Gateway
Messaging	Not established here from accessible record	Kafka producer/consumer implemented
Prediction	Institutional-support focus	Attendance prediction service
Campus modules	Institutional support	Academic, resource, event, transport,
AI workflow	AI-agent support	LangGraph intent routing + prediction/service
Current evidence	Conference publication	Working UI and code-level prototype evidence

The main areas in which EduFusion AI is stronger than the base paper for the present project objective are breadth of operational modules, explicit role separation, independently deployable backend services, an event-driven integration path, and a measurable student-risk prediction component. It also provides a visible administrative/faculty/student portal rather than limiting the system to an agent interface. These advantages are implementation breadth advantages, not a claim of superior AI accuracy.

The base paper remains a useful reference because it motivates the use of an institutional AI agent. EduFusion AI differs by making the AI component one layer in a larger campus operating platform. The strongest future improvement is to turn the present LangGraph workflow into genuinely specialized agents that call authorized microservice tools, then evaluate the complete workflows with repeated latency, reliability, prediction, and intervention measurements.

X. DISCUSSION

The current prototype demonstrates that a smart-campus system can be assembled from independently testable components without forcing all functions into a monolithic application. The service boundaries make it possible to modify attendance or resource logic without rewriting the entire portal. Kafka adds asynchronous decoupling, while FastAPI isolates the Python AI environment from Java services.

The most important methodological observation is that the AI layer should be evaluated according to what it actually executes. The project contains a Random Forest training script, but the deployed prediction endpoint uses a deterministic weighted heuristic. Likewise, the LangGraph service currently uses intent-routing nodes and a real prediction-service call for prediction requests, while some student actions are represented by mock service results. Reporting these boundaries makes the work more reproducible and identifies the exact engineering steps required to turn the prototype into a research-grade experimental system.

A complete future evaluation should use an approved institutional dataset or a clearly documented synthetic dataset with realistic relationships across attendance, assessment, events, room bookings, resources, transport, and infrastructure. Model evaluation should include precision, recall, F1, confusion matrix, calibration, and cross-validation. Service evaluation should include latency, throughput, availability, error rate, Kafka lag, workflow completion rate, and resource-allocation constraint satisfaction.

XI. LIMITATIONS

- The current predictive dataset is synthetic and generated from a hand-designed rule; therefore, the 88.5% accuracy does not establish real-world generalization.
- The deployed prediction endpoint does not currently load the saved Random Forest model and instead uses deterministic weighted inference.
- The LangGraph agent contains intent routing and prediction integration, while some service actions remain mocked or simplified.
- Dashboard values visible in the supplied prototype screenshots are demonstration values and should not be described as real institutional measurements.
- End-to-end Kafka reliability, load testing, security penetration testing, and multi-agent workflow benchmarking require additional experiments.

XII. CONCLUSION

EduFusion AI presents an implementable architecture for intelligent campus management that combines a role-aware React portal, Spring Boot microservices, PostgreSQL, Apache Kafka, FastAPI AI services, LangGraph orchestration, and student-risk prediction. The current prototype already demonstrates a broad set of operational modules and role-specific interfaces. The offline Random Forest experiment achieves 88.5% test accuracy on a reproducible 1,000-record synthetic dataset, while the deployed endpoint provides deterministic risk and final-score inference. The paper deliberately separates these two results to maintain scientific accuracy.

Relative to the 2025 CAMPUS-ONE conference work, EduFusion AI's primary advantage for this project is its wider operational scope and explicit microservice/event infrastructure. The next research stage should replace synthetic labels with an approved dataset, load and benchmark the trained model in the prediction service, replace mocked agent actions with authorized microservice tools, and evaluate complete closed-loop workflows. These changes would move EduFusion AI from a strong integrated prototype toward a rigorously evaluated smart-campus research platform.

REFERENCES

[1] K. M. D., B. J. Dsouza, A. B., C. B., et al., “CAMPUS-ONE—An AI Agent for Institutional Support,” in Proc. 2025 6th Int. Conf. IoT Based Control Networks and Intelligent Systems (ICICNIS), 2025, pp. 2181–2186, doi: 10.1109/ICICNIS66685.2025.11315812.

[2] “SAPP: Student Academic Performance Predictor,” in Proc. 2025 IEEE Global Engineering Education Conf. (EDUCON), London, U.K., Apr. 2025, doi: 10.1109/EDUCON62633.2025.11016395.

[3] “Smart Campus, Smarter Students: Enhancing Learning Outcomes through AI-IoT-Enabled Sustainable Spaces,” in Proc. 2025 10th Int. STEM Education Conf. (iSTEM-Ed), Pattaya, Thailand, 2025, doi: 10.1109/iSTEM-Ed65612.2025.11129276.

[4] “AI Powered Academic Assistant Using Conversational AI,” in Proc. 2025 3rd Int. Conf. on Advancements in Electrical, Electronics, Communication, Computing and Automation (ICAECA), Coimbatore, India, Apr. 2025, doi: 10.1109/ICAECA63854.2025.11012528.

[5] “Student Performance Prediction System Using Machine Learning,” in Proc. 2025 IEEE Int. Conf. on Emerging Technologies and Applications (MPSec ICETA), Gwalior, India, Feb. 2025, doi: 10.1109/MPSecICETA64837.2025.11118334.

[6] “Predicting Student Performance Using Deep Learning and Machine Learning Techniques,” in Proc. 2025 1st Int. Conf. on Computational Intelligence Approaches and Applications (ICCIAA), Amman, Jordan, Apr. 2025, doi: 10.1109/ICCIAA65327.2025.11013240.

[7] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), Athens, Greece, 2011.

[8] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

Prototype manuscript prepared in IEEE conference-style two-column format. Results are explicitly separated into measured model results and illustrative UI validation.